Matrices – Sudoku

Erkki Kaila, Antti Laaksonen, Matti Luukkainen and Arto Hellas are the original creators of these sudoku exercises. They have shared the original version under Creative Commons BY-NC-SA 4.0 licence. Kari Silpiö has adapted / modified these exercises. These modified sudoku exercises are shared under Creative Commons BY-NC-SA 4.0 licence.

We can use the matrix data structure when we are creating various games.

5	6	8		9	3			7
3	4				7			5
	9	7	5		4	6		3
7				1	2	5		
	1	9			8	7	6	
	8		7				3	
	2			3	5	8		
	5	1	2		6	3		
			9					

For example, we can represent the above grid of the sudoku game as a matrix as below. Zero represents an empty square.

```
sudoku = [
    [5, 6, 8, 0, 9, 3, 0, 0, 7],
    [3, 4, 0, 0, 0, 7, 0, 0, 5],
    [0, 9, 7, 5, 0, 4, 6, 0, 3],
    [7, 0, 0, 0, 1, 2, 5, 0, 0],
    [0, 1, 9, 0, 0, 8, 7, 6, 0],
    [0, 8, 0, 7, 0, 0, 0, 3, 0],
    [0, 2, 0, 0, 3, 5, 8, 0, 0],
    [0, 5, 1, 2, 0, 6, 3, 0, 0],
    [0, 0, 0, 9, 0, 0, 0, 0, 0],
]
```

Then we can print the grid, for example, as follows:

```
for row in sudoku:
    for square in row:
        if square > 0:
            print(f"{square}", end=" ")
        else:
            print("_", end=" ")
    print()
```

The printout of the grid is as below.

```
5 6 8 _ 9 3 _ _ 7
3 4 _ _ _ 7 _ _ 5
_ 9 7 5 _ 4 6 _ 3
7 _ _ _ 1 2 5 _ _
_ 1 9 _ _ 8 7 6 _
_ 8 _ 7 _ _ _ 3 _
_ 2 _ _ 3 5 8 _ _
_ 5 1 2 _ 6 3 _ _
_ _ _ 9 _ _ _ _ _
```

SUDOKU EXERCISES

NB! In this set of sudoku exercises,

- You will be asked to write a specific function in each exercise.
- In Viope, there are ***unit tests***, which will ***test only your function*** instead of testing your whole program.

Therefore, there are no example outputs given in this set of exercises. Instead of your program's output, Viope will show the unit test's output when you run your program in Viope.

- To prevent your whole program to be run when only one of its functions is being tested, please call your own main function in your program as below.

```
if __name__ == "__main__":
    main()
```

- **Hint**: We have already used **set**() to eliminate duplicates... maybe you find some use for it again

Sudoku Check Row

Create a program called **sudoku_check_row**. The program should have a function named **row_ok** that takes *sudoku grid* (matrix) and *row index* (int) as arguments.

You can suppose that each item on a row in the matrix is a digit between 0 and 9. Zero represents an empty square in a sudoku grid. That is, there can be several zeros on a row until the sudoku is solved.

The **row_ok** function should return **True** if the row is ok. Otherwise, it should return **False**. **NB!** The function should not modify the sudoku grid.

A row is ok if each of the digits between 1 and 9 appears zero or one times on the row. The row is not ok if any of the digits between 1 and 9 appears more than once on the row.

In the sudoku grid below, *the second row is not ok* because the digit 2 appears twice on the row. All the other rows are ok.

NB! Please copy/paste the code of the main function below to your program.

```
def main():
    sudoku = [
        [9, 0, 0, 0, 8, 0, 3, 0, 0],
        [2, 0, 0, 2, 5, 0, 7, 0, 0],
        [0, 2, 0, 3, 0, 0, 0, 0, 4],
        [2, 9, 4, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 7, 3, 0, 5, 6, 0],
        [7, 0, 5, 0, 6, 0, 4, 0, 0],
        [0, 0, 7, 8, 0, 3, 9, 0, 0],
        [0, 0, 1, 0, 0, 0, 0, 0, 3],
        [3, 0, 0, 0, 0, 0, 0, 0, 2]
    ]

    print(row_ok(sudoku, row_index=0))    # True
    print(row_ok(sudoku, row_index=1))    # False
    print(row_ok(sudoku, row_index=8))    # True

if __name__ == "__main__":
    main()
```

Sudoku Check Column

Create a program called **sudoku_check_column**. The program should have a function named **column_ok** that takes *sudoku grid* (matrix) and *column index* (int) as arguments.

The **column_ok** function should return **True** if the row is ok. Otherwise, it should return **False**. **NB!** The function should not modify the sudoku grid.

A column is ok if each of the digits between 1 and 9 appears zero or one times in the column. The column is not ok if any of the digits between 1 and 9 appears more than once in the column.

In the sudoku grid below, *the first column is not ok* because the digit 2 appears twice in the column. All the other columns are ok.

NB! Please copy/paste the code of the main function below to your program.

```
def main():
    sudoku = [
        [9, 0, 0, 0, 8, 0, 3, 0, 0],
        [2, 0, 0, 2, 5, 0, 7, 0, 0],
        [0, 2, 0, 3, 0, 0, 0, 0, 4],
        [2, 9, 4, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 7, 3, 0, 5, 6, 0],
        [7, 0, 5, 0, 6, 0, 4, 0, 0],
        [0, 0, 7, 8, 0, 3, 9, 0, 0],
        [0, 0, 1, 0, 0, 0, 0, 0, 3],
        [3, 0, 0, 0, 0, 0, 0, 0, 2]
    ]

    print(column_ok(sudoku, column_index=0))    # False
    print(column_ok(sudoku, column_index=1))    # True
    print(column_ok(sudoku, column_index=8))    # True

if __name__ == "__main__":
    main()
```

Sudoku Check Block

Create a program called **sudoku_check_block**. The program should have a function named **block_ok** that takes *sudoku grid* (matrix), *row index* (int), and *column index* (int) as arguments.

The **block_ok** function should return **True** if the 3x3 block is ok. Otherwise, it should return **False**. **NB!** The function should not modify the sudoku grid.

A 3x3 block is ok if each of the digits between 1 and 9 appears zero or one times in the block. The block is not ok if any of the digits between 1 and 9 appears more than once in the block.

In a sudoku grid, the 3x3 blocks begin at the indexes (row_index, column_index)

(0, 0), (0, 3), (0, 6), (3, 0), (3, 3), (3, 6), (6, 0), (6, 3), (6, 6)

NB! If the **block_ok** function is called with any other combination of row and column, it should return **False**.

NB! Please copy/paste the code of the main function below to your program.

```
def main():
    sudoku = [
        [9, 0, 0, 0, 8, 0, 3, 0, 0],
        [2, 0, 0, 2, 5, 0, 7, 0, 0],
        [0, 2, 0, 3, 0, 0, 0, 0, 4],
        [2, 9, 4, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 7, 3, 0, 5, 6, 0],
        [7, 0, 5, 0, 6, 0, 4, 0, 6],
        [0, 0, 7, 8, 0, 3, 9, 0, 0],
        [0, 0, 1, 0, 0, 0, 0, 0, 3],
        [3, 0, 0, 0, 0, 0, 0, 0, 2]
    ]

    print(block_ok(sudoku, row_index=0, column_index=0))    # False
    print(block_ok(sudoku, row_index=3, column_index=6))    # False
    print(block_ok(sudoku, row_index=6, column_index=3))    # True

if __name__ == "__main__":
    main()
```

The first function call above should check the 3x3 block at the indexes (0, 0).

```
9 0 0
2 0 0
0 2 0
```

The second function call above should check the 3x3 block at the indexes (3, 6).

```
0 0 0
5 6 0
4 0 6
```

The third function call above should check the 3x3 block at the indexes (6, 3).

```
8 0 3
0 0 0
0 0 0
```

Sudoku Check Grid

Create a program called **sudoku_check_grid**. The program should have a function named **grid_ok** that takes *sudoku grid* (matrix) as argument. The function should use the functions from the three previous sudoku exercises to determine whether the complete sudoku grid is ok.

The **grid_ok** function should check each of the nine rows, nine columns, and nine 3x3 blocks in the sudoku grid. If all of them are ok, the function should return **True**. Otherwise, the function should return **False**.

NB! First, copy/paste the functions (row_ok, column_ok, and block_ok) from your previous exercise solutions to this program. Then copy/paste the code of the main function below to your program. Finally, write the **grid_ok** function.

```
def main():
    sudoku_1 = [
        [9, 0, 0, 0, 8, 0, 3, 0, 0],
        [2, 0, 0, 2, 5, 0, 7, 0, 0],
        [0, 2, 0, 3, 0, 0, 0, 0, 4],
        [2, 9, 4, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 7, 3, 0, 5, 6, 0],
        [7, 0, 5, 0, 6, 0, 4, 0, 0],
        [0, 0, 7, 8, 0, 3, 9, 0, 0],
        [0, 0, 1, 0, 0, 0, 0, 0, 3],
        [3, 0, 0, 0, 0, 0, 0, 0, 2]
    ]

    sudoku_2 = [
        [2, 6, 7, 8, 3, 9, 5, 0, 4],
        [9, 0, 3, 5, 1, 0, 6, 0, 0],
        [0, 5, 1, 6, 0, 0, 8, 3, 9],
        [5, 1, 9, 0, 4, 6, 3, 2, 8],
        [8, 0, 2, 1, 0, 5, 7, 0, 6],
        [6, 7, 4, 3, 2, 0, 0, 0, 5],
        [0, 0, 0, 4, 5, 7, 2, 6, 3],
        [3, 2, 0, 0, 8, 0, 0, 5, 7],
        [7, 4, 5, 0, 0, 3, 9, 0, 1]
    ]

    print(grid_ok(sudoku_1))    # False
    print(grid_ok(sudoku_2))    # True

if __name__ == "__main__":
    main()
```